

คณิตศาสตร์พื้นฐาน สำหรับ Options Trading

Part V: Optimization — หาคำตอบที่ดีที่สุด

บทที่ 10-11: Linear Programming, Simplex, Dual, Integer LP,
Gradient Descent, Lagrange, Convex Optimization, QP

บทที่ 10: Linear Programming

อุปมา: ร้านขนมที่มีส่วนผสมจำกัด

สมมติเปิดร้านขนม ทำได้ 2 อย่าง: **ขนมปัง A** (กำไร 8 บาท) และ **ขนมปัง B** (กำไร 5 บาท)

แต่มีข้อจำกัด: แป้ง 20 ถ้วย (A ใช้ 5 ถ้วย, B ใช้ 2 ถ้วย) และ เวลา 5 ชั่วโมง (แต่ละชิ้น 1 ชั่วโมง)

คำถาม: ทำ A กี่ชิ้น B กี่ชิ้น *จึงจะกำไรมากที่สุด?* — นี่คือ Linear Programming!

ทรัพยากรจำกัด

→

ตั้ง Constraints

→

เขียน Objective

→

หาจุด Optimal

LP = หาค่า maximum/minimum ของฟังก์ชันเชิงเส้น ภายใต้เงื่อนไข (constraints) ที่เป็นเชิงเส้น

10.1 แนวคิดและการตั้งโจทย์

$$\begin{aligned} \text{Maximize} \quad & z = c_1x_1 + c_2x_2 \\ \text{subject to} \quad & a_{11}x_1 + a_{12}x_2 \leq b_1 \\ & a_{21}x_1 + a_{22}x_2 \leq b_2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

ทำไมสมการถึงเขียนแบบนี้?

c_1, c_2 = "กำไรต่อหน่วย" ของแต่ละตัวเลือก → เป้าหมายคือ maximize ผลรวม

a, b = "ส่วนผสมที่ใช้" vs "ส่วนผสมที่มี" → ข้อจำกัดทางกายภาพ

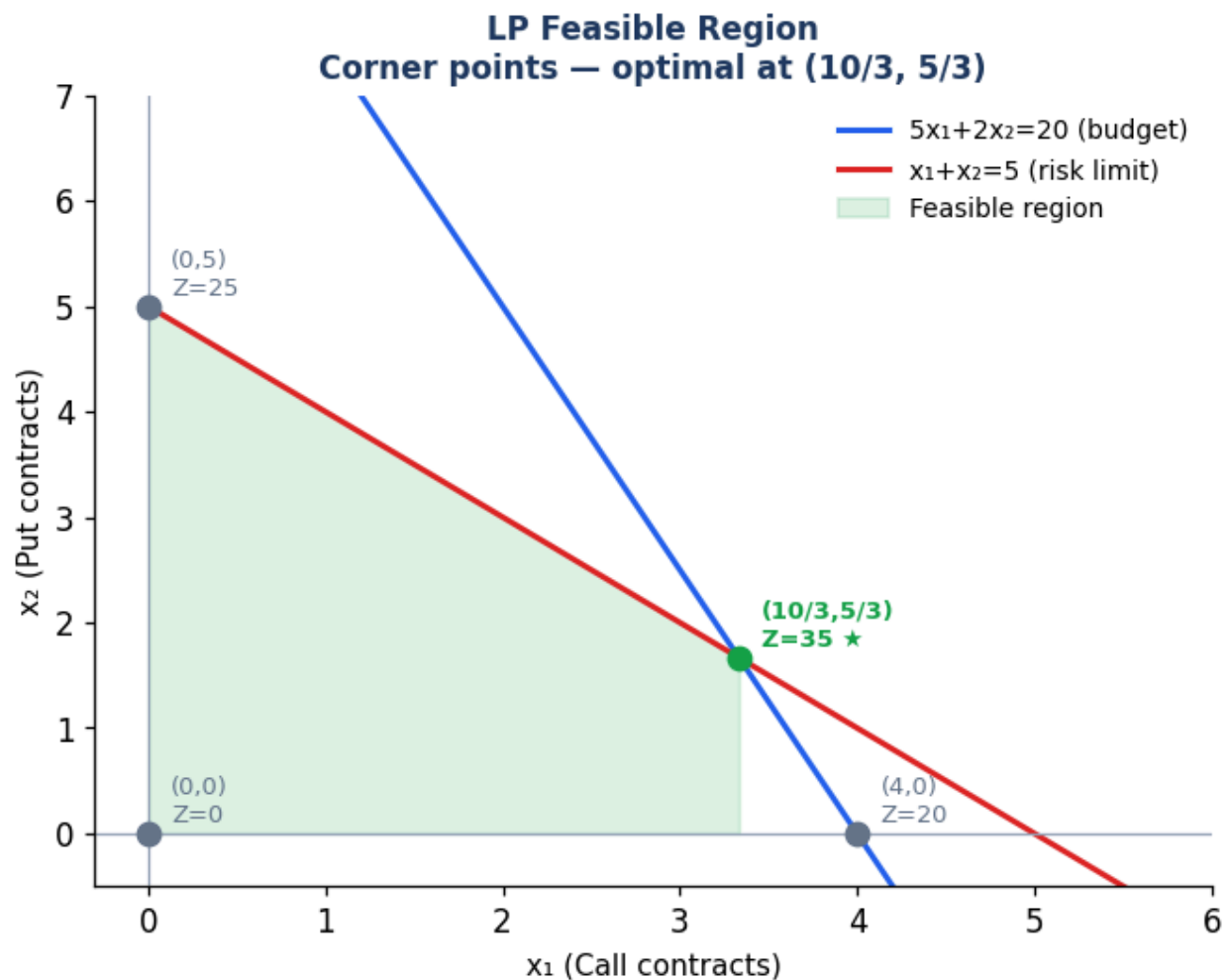
ทุก constraint เป็น "เส้นตรง" → ทำให้ feasible region เป็น "รูปหลายเหลี่ยม" → คำตอบอยู่ที่มุม!

ตัวอย่าง: จัดพอร์ต Options**Maximize** Expected Payoff = $8x_1 + 5x_2$ $(x_1 = \text{จำนวน Call } K=100, x_2 = \text{จำนวน Call } K=110)$ **Subject to:**Premium: $5x_1 + 2x_2 \leq 20$ (budget ฿20)Risk: $x_1 + x_2 \leq 5$ (ไม่เกิน 5 สัญญา) $x_1, x_2 \geq 0$ **10.2 Graphical Method****อุปมา: แผนที่ที่มีรั้วกัน**

นึกภาพแผนที่ 2 มิติ Constraint แต่ละข้อคือ "รั้ว" ที่กั้น พื้นที่ที่เหลือ (ข้างใน) คือ **Feasible Region** — บริเวณที่ทำได้จริง

Objective Function คือ "เส้นระดับความสูง" ที่ขนานกัน → เลื่อนเส้นออกไปเรื่อยๆ จนชนรั้ว → จุดนั้นคือ optimal!

สำหรับ 2 ตัวแปร แก้ได้ด้วยกราฟ: วาด constraints → หา feasible region → เช็ก corner points

LP feasible region with corner points; optimal at $(10/3, 5/3)$ **Corner Point Theorem**

คำตอบ optimal ของ LP อยู่ที่มุม (corner) ของ feasible region เสมอ!

ดังนั้นเช็คแค่ทุก corner → เลือกค่า z สูงสุด

1 หา **corner points** จากจุดตัดของ constraints:

- $(0,0)$ — origin
- $(0,5)$ — constraint 2 ตัด y-axis: $x_1=0 \rightarrow x_2=5$
- $(10/3, 5/3)$ — constraints 1 และ 2 ตัดกัน: แก่ระบบ $5x_1+2x_2=20, x_1+x_2=5$
- $(4,0)$ — constraint 1 ตัด x-axis: $x_2=0 \rightarrow 5x_1=20 \rightarrow x_1=4$

2 คำนวณ z ที่แต่ละ **corner**:

- $z(0,0) = 8(0)+5(0) = 0$
- $z(0,5) = 8(0)+5(5) = 25$
- $z(10/3, 5/3) = 8(10/3)+5(5/3) = 80/3+25/3 = 105/3 = 35 \leftarrow \text{Maximum!}$
- $z(4,0) = 8(4)+5(0) = 32$

3 สรุป: ซื้อ Call $K=100$ จำนวน 3.33 สัญญา + Call $K=110$ จำนวน 1.67 สัญญา $\rightarrow$ Payoff = ฿35

10.3 Simplex Method

อุปมา: เดินป่าในความมืด หาจุดสูงสุด

ลองนึกถึงการเดินป่าตอนกลางคืน มองไม่เห็นทิวทัศน์ทั้งหมด แต่ *รู้สึกได้* ว่าทิศไหนลาดขึ้นชัน

วิธี Simplex: ยืนอยู่ที่มุมหนึ่ง $\rightarrow$ มองไปมุมใกล้ๆ $\rightarrow$ เดินไปมุมที่สูงกว่า $\rightarrow$ ทำซ้ำจนไม่มีมุมที่สูงกว่า

ไม่ต้องสำรวจทุกมุม! ในทางปฏิบัติ Simplex เร็วมากแม้มีหลายร้อยตัวแปร

เมื่อมีหลายตัวแปร (ใช้กราฟไม่ได้) ใช้ **Simplex Algorithm**:

1. เริ่มที่ corner หนึ่ง (มักเป็นจุด origin)
2. มองไปรอบๆ ว่า corner ไหนดีกว่า (ดู "reduced cost")
3. เดินไปที่ corner ที่ดีขึ้นมากที่สุด (pivot)
4. ทำซ้ำจนไม่มี corner ที่ดีกว่า $\rightarrow$ **optimal!**

Corner Point Theorem

$\rightarrow$

คำตอบอยู่ที่มุม

$\rightarrow$

เดินจากมุมสู่มุมที่ดีขึ้น

$\rightarrow$

หยุดเมื่อไม่มีมุมดีกว่า

$\rightarrow$

Optimal!

Simplex ไม่ต้องจำขั้นตอน แค่เข้าใจหลักการ

"เดินไปตาม edge ของ feasible region จาก corner สู่ corner ที่ดีขึ้นเรื่อยๆ"
 ในทางปฏิบัติใช้ software (Excel Solver, Python scipy.optimize.linprog)

10.4 Dual Problem & Shadow Price

อุปมา: "ถ้าได้วัตถุดิบเพิ่ม 1 หน่วย กำไรเพิ่มเท่าไร?"

เจ้าของร้านขนมถามว่า "ถ้าซื้อแป้งเพิ่ม 1 ถ้วย (ราคา ฿?) กำไรจะเพิ่มขึ้นเท่าไร?" — นั่นคือ Shadow Price!

Shadow Price = "ราคาเงา" ของ constraint — บอกว่า constraint ไหน "แพง" ที่สุด ควรไปผ่อนคลายก่อน

ทุก LP (Primal) มีคู่ (Dual) — ให้มุมมองเพิ่มเติม:

$$\text{Primal: } \max \mathbf{c}^T \mathbf{x} \text{ s.t. } \mathbf{Ax} \leq \mathbf{b}, \mathbf{x} \geq 0$$

$$\text{Dual: } \min \mathbf{b}^T \mathbf{y} \text{ s.t. } \mathbf{A}^T \mathbf{y} \geq \mathbf{c}, \mathbf{y} \geq 0 \quad \Rightarrow \quad \text{Strong Duality: ค่าเท่ากันที่ optimal}$$

ทำไม Dual มีอยู่?

Primal ถามว่า "จัด resource ยังไงให้ได้กำไรสูงสุด?"

Dual ถามว่า "resource แต่ละอย่างมีมูลค่าเท่าไร?" (y^* = shadow prices)

Strong Duality: ทั้งสองคำถามมีคำตอบที่สอดคล้องกันที่ optimal → สามารถใช้ Dual แทน Primal ได้เมื่อสะดวกกว่า

Shadow Price (y^*) = "ถ้าเพิ่ม resource (b) อีก 1 หน่วย ค่า objective จะเพิ่มเท่าไร?"

- 1 จากตัวอย่าง Options: ที่ optimal (10/3, 5/3) — budget constraint: $5x_1 + 2x_2 = 20$ (active)
- 2 Shadow price ของ budget constraint = ฿1/บาท budget (จาก Dual: $y_1 = 1, y_2 = 3$)
- 3 ถ้า budget เพิ่มจาก ฿20 → ฿21 → Expected Payoff เพิ่มจาก ฿35 → ฿36
- 4 ถ้า risk constraint active: shadow price บอกว่า "การเพิ่ม 1 สัญญา" มีมูลค่าเท่าไร

เชื่อมกับ Options

Shadow Price ของ budget constraint = "ถ้ามีเงินเพิ่มอีก \$1 expected payoff จะเพิ่มเท่าไร?"
ใช้ตัดสินใจว่าควร "ขยาย budget" หรือ "ลด risk limit" — constraint ไหน "แพง" กว่า



Trading Desk: Shadow Price = "ราคา" ของ Risk Limit แต่ละข้อ

Fund manager ที่มี position limits ใช้ Dual / Shadow Price เพื่อ:

- **VaR constraint:** shadow price = "ถ้าเพิ่ม VaR limit \$1M → PnL potential เพิ่มเท่าไร?"
- **Capital constraint:** "margin ที่ต้องวาง \$1M มีค่า opportunity cost เท่าไร?"
- **Concentration limit:** "ถ้า sector limit ถูก relax 1% → เพิ่ม return ได้เท่าไร?"

Shadow price เป็นเครื่องมือช่วย **argue กับ risk manager** ว่า limit ไหนควรปรับก่อน

10.5 Sensitivity Analysis

ถ้า parameter เปลี่ยน คำตอบเปลี่ยนไหม? — สำคัญเพราะ input มักไม่แน่นอน

- **Coefficient ใน objective เปลี่ยน:** ถ้า expected payoff ของ Call เปลี่ยน → basis ยังเหมือนเดิมไหม? (มี "allowable range")
- **RHS ของ constraint เปลี่ยน:** ถ้า budget เพิ่ม → ใช้ shadow price คำนวณได้ทันที (ดูราบที่อยู่ใน range)

Sensitivity ใน Options Trading

Greeks ของ Options ก็คือ sensitivity analysis! Delta = sensitivity ของ option price ต่อ S, Vega = sensitivity ต่อ σ

Simplex sensitivity = sensitivity ของ optimal solution ต่อ parameters ของโจทย์ LP

10.6 Integer Linear Programming (ILP)

อุปมา: ซื้อตัวเครื่องบินครึ่งใบไม่ได้

LP อาจตอบว่า "ควรซื้อ 3.7 สัญญา" — แต่สัญญา Options ซื้อเป็นจำนวนเต็มเท่านั้น! (1 contract = 100 shares)

ILP แก้ปัญหานี้โดยบังคับให้ตัวแปรเป็นจำนวนเต็ม

ILP = LP ที่ตัวแปรต้องเป็นจำนวนเต็ม

$$\max \mathbf{c}^T \mathbf{x} \text{ s.t. } A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq 0, \mathbf{x} \in \mathbb{Z}^n$$

แก้ LP Relaxation

→

ตัวแปรไม่เต็ม?

→

Branch ($x \leq 3$ หรือ $x \geq 4$)

→

แก้ใหม่แต่ละ branch

→

Integer Solution

ILP ยากกว่า LP มาก!

LP แก้ได้ใน polynomial time แต่ ILP เป็น NP-hard → ไม่มี algorithm ที่เร็วเสมอ

วิธีแก้: **Branch and Bound** — แก้ LP relaxation ก่อน แล้ว branch ที่ตัวแปรที่ไม่เป็นจำนวนเต็ม

เชื่อมกับ Options

Options ซื้อเป็น **สัญญา** (1 contract = 100 หุ้น) → ต้องเป็นจำนวนเต็ม!

ตัวอย่าง: เลือก combination ของ option contracts ที่ maximize payoff-to-risk ภายใต้ budget constraint → ILP problem

บทที่ 11: Nonlinear Optimization

อุปมา: โลกจริงไม่ใช่เส้นตรง

Portfolio risk ไม่ใช่ linear — เพิ่ม 2 เท่าของสินทรัพย์ไม่ได้เพิ่ม risk 2 เท่า (เพราะ diversification!)

Nonlinear optimization จัดการกับฟังก์ชันที่ "โค้ง" — อย่าง Portfolio Variance = $w^T \Sigma w$ ซึ่งเป็น *quadratic* (โค้งพาราโบลา)

โลกจริงไม่เป็นเส้นตรง — Portfolio Optimization ของ Markowitz เป็น **Quadratic** ไม่ใช่ Linear

11.1 Unconstrained Optimization

หา min/max ของ $f(x)$ โดยไม่มี constraints:

$$f'(x^*) = 0 \quad \begin{cases} f''(x^*) > 0 & \Rightarrow \text{minimum} \\ f''(x^*) < 0 & \Rightarrow \text{maximum} \end{cases}$$

ทำไม $f'=0$ คือ optimal?

ถ้า slope $\neq 0$ หมายความว่าเดินไปอีกนิดจะ "ดีขึ้น" → ยังไม่ optimal!

ที่ optimal จริง: ทุกทิศทางจาก x^* จะ "แย่ลง" → slope ต้องเป็น 0 (หรือ gradient = 0 สำหรับหลายตัวแปร)

11.2 Gradient Descent — "เดินลงเขา"

อุปมา: ตาบอด ลงเขาในหมอก

ลองนึกถึงการลงเขาตอนหมอกลง มองไม่เห็นทาง แต่รู้สึกได้ว่าทิศไหน "ลาดชันลงมากที่สุด"

วิธี Gradient Descent: คำนวณทิศที่ "ชันขึ้น" (gradient) แล้วเดินสวนทาง → ลงเร็วที่สุด

ขนาดก้าว (α = learning rate): เล็กเกินไป = ช้า, ใหญ่เกินไป = กระโดดข้ามหุบเขา!

$$x_{n+1} = x_n - \alpha \nabla f(x_n)$$

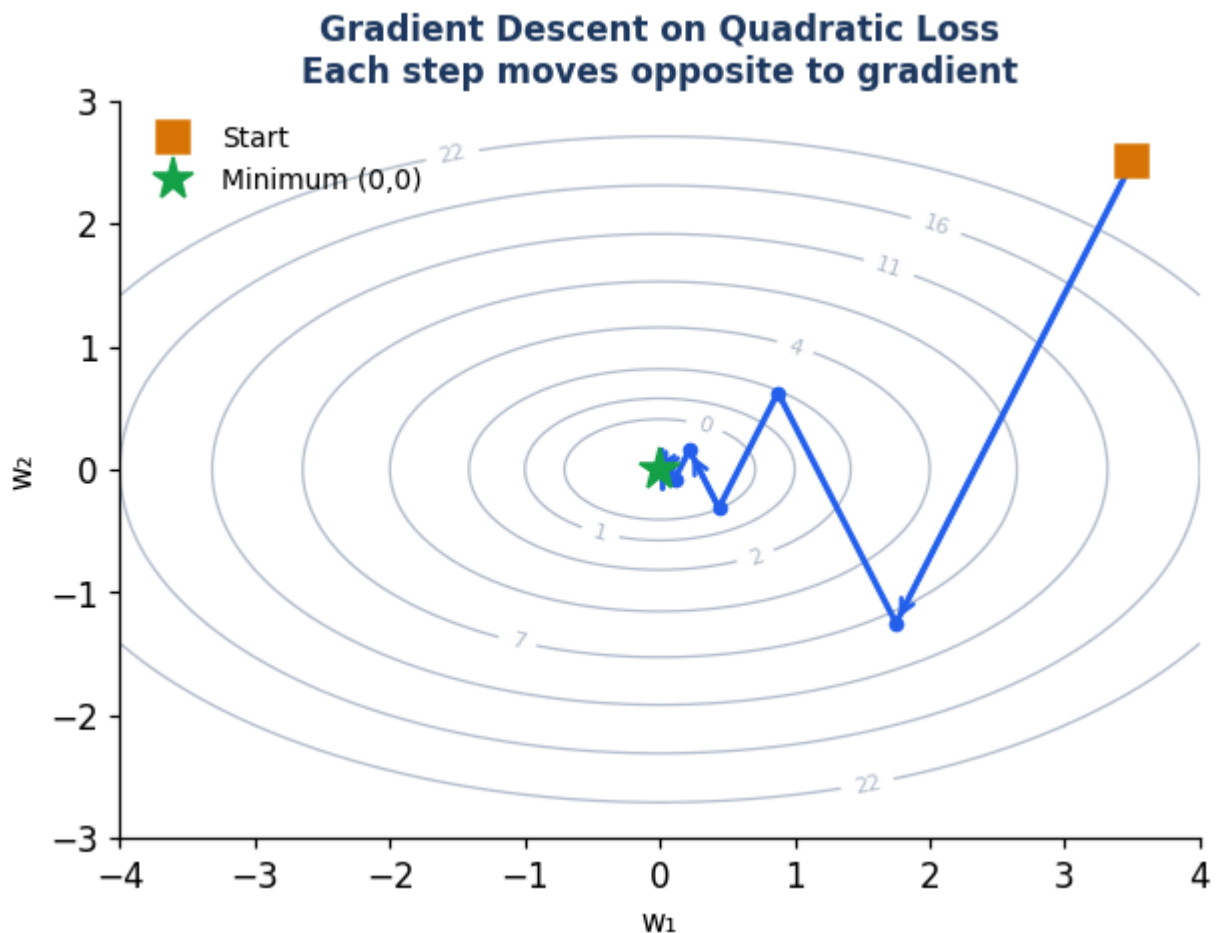
α = learning rate (ขนาดก้าว), ∇f = gradient (ทิศที่ f เพิ่มขึ้นเร็วสุด) → เดินสวนทาง = ลดเร็วสุด

คำนวณ ∇f (gradient) →เดินสวนทาง ($-\alpha \nabla f$) →Update x →

ทำซ้ำ →

 $|\nabla f| \approx 0 \rightarrow$ หยุด

- 1 โจทย์: Minimize $f(x) = x^2$ โดยใช้ Gradient Descent, $\alpha=0.3$, เริ่มที่ $x_0=4$
- 2 $f'(x) = 2x$ (gradient)
- 3 Iteration 1: $x_1 = 4 - 0.3 \times (2 \times 4) = 4 - 2.4 = \mathbf{1.6}$
- 4 Iteration 2: $x_2 = 1.6 - 0.3 \times (2 \times 1.6) = 1.6 - 0.96 = \mathbf{0.64}$
- 5 Iteration 3: $x_3 = 0.64 - 0.3 \times (2 \times 0.64) = 0.64 - 0.384 = \mathbf{0.256}$
- 6 เข้าใกล้ 0 (คำตอบจริง) ทุก iteration! หลังหลาย iteration $\rightarrow x \rightarrow 0$



Gradient descent on quadratic bowl — each step moves opposite to gradient

เชื่อมกับ Options

ใช้ Gradient Descent สำหรับ:

- **Model Calibration:** หา parameters ที่ทำให้ model price $\approx$ market price
- **Volatility Surface Fitting:** minimize ผลต่างระหว่าง model vol vs market vol

11.3 Lagrange Multipliers — Optimization ที่มี Constraints

อุปมา: เดินบนถนน (constraint) หาจุดที่สูงสุด

ถ้าต้องเดินอยู่บนถนน (constraint $g=0$) แต่อยากหาจุดที่ "สูงที่สุด" (maximize f)

จุด optimal คือจุดที่ "ทิศทางขึ้นสูง" ชี้ *ตั้งฉากกับถนน* — ถ้าไม่ตั้งฉาก แปลว่าเดินต่อบนถนนแล้วจะสูงขึ้นได้อีก!

Lagrange Multiplier (λ) = อัตราส่วนระหว่าง gradient ของ f กับ gradient ของ g ที่จุด optimal

หา min/max ของ $f(x)$ ภายใต้เงื่อนไข $g(x) = 0$:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda g(x) \quad \frac{\partial \mathcal{L}}{\partial x} = 0, \quad \frac{\partial \mathcal{L}}{\partial \lambda} = 0$$

$$\nabla f = \lambda \nabla g \quad (\text{gradients parallel at optimum})$$

- 1 **Markowitz Problem:** Minimize $\sigma^2_p = w^T \Sigma w$ s.t. $w^T \mu = r^*$ (target) และ $w^T 1 = 1$
- 2 Lagrangian: $L = w^T \Sigma w - \lambda_1(w^T \mu - r^*) - \lambda_2(w^T 1 - 1)$
- 3 $\partial L / \partial w = 0 \rightarrow 2 \Sigma w - \lambda_1 \mu - \lambda_2 1 = 0 \rightarrow w^* = \Sigma^{-1}(\lambda_1 \mu + \lambda_2 1) / 2$
- 4 ปรับ λ_1, λ_2 ให้ตรงกับ constraints $\rightarrow$ ได้ efficient portfolio!
- 5 ทำสำหรับ r^* ต่างๆ $\rightarrow$ ได้ **Efficient Frontier** (เส้นโค้ง risk vs return)

เชื่อมกับ Options — Markowitz Portfolio Optimization!

Minimize Portfolio Risk: $\sigma^2_p = w^T \Sigma w$

Subject to: $w^T \mu = \text{target return}, w^T 1 = 1$

ใช้ Lagrange: $L = w^T \Sigma w - \lambda_1(w^T \mu - r^*) - \lambda_2(w^T 1 - 1)$

$\rightarrow$ หา **Efficient Frontier** (เส้นโค้งของ risk vs return ที่ดีที่สุด)

11.4 KKT Conditions — Optimization ที่มี Inequality Constraints

อุปมา: Lagrange แก่ "ต้องอยู่บนเส้น" แต่ชีวิตจริงคือ "ห้ามเกินเพดาน" 🔍

Lagrange จัดการได้แค่ constraint แบบ **equality** ($g=0$) — เหมือนบังคับว่า "ต้องเดินอยู่บนถนนเส้นนี้เป๊ะๆ"

แต่ชีวิตจริงของพอร์ตคือ constraint แบบ **inequality**: "ห้าม short" ($w \geq 0$), "ห้าม position เกิน 10%" ($w \leq 0.1$) — เป็น "เพดาน" และ "พื้น" ไม่ใช่เส้นเดียว

KKT (Karush-Kuhn-Tucker) = Lagrange เวอร์ชันที่รับ inequality ได้ — กฎเกณฑ์สำคัญสู่การจัดพอร์ต option จริงๆ

หา min ของ $f(x)$ ภายใต้เงื่อนไข inequality $g_i(x) \leq 0$ — เงื่อนไข KKT มี 4 ข้อ:

$$\min f(\mathbf{x}) \quad \text{s.t.} \quad g_i(\mathbf{x}) \leq 0$$

1. Stationarity: $\nabla f + \sum_i \mu_i \nabla g_i = 0$
2. Primal feasibility: $g_i(\mathbf{x}) \leq 0$
3. Dual feasibility: $\mu_i \geq 0$
4. Complementary slackness: $\mu_i \cdot g_i(\mathbf{x}) = 0$

หัวใจของ KKT คือ Complementary Slackness

ข้อ 4 ($\mu_i \cdot g_i = 0$) บอกว่าแต่ละ constraint ต้องอยู่ในสถานะใดสถานะหนึ่งเท่านั้น:

- **Active (binding):** $g_i=0$ และ $\mu_i>0 \rightarrow$ constraint "กีด" — เราชนเพดาน/พื้นพอดี เช่น โดนพื้น no-short ($w=0$) ดึงไว้ทำให้ μ มีค่า (shadow price > 0)

- **Inactive (slack):** $g_i<0$ และ $\mu_i=0 \rightarrow$ constraint "ไม่กีด" — เรายังห่างจากเพดาน constraint นี้จึงไม่มีผล ($\mu=0$)

พูดอีกอย่าง: ถ้า constraint ไม่ binding มันก็ไม่มี shadow price — μ จะเป็น 0 โดยอัตโนมัติ นี่คือโอเคเดียวที่ทำให้ KKT จัดการ inequality ได้

ตัวอย่าง: Minimize $f(w)=w^2$ s.t. $w \geq 1$

เขียน constraint ในรูป $g_i \leq 0$: $w \geq 1 \iff g(w) = 1-w \leq 0$

สังเกตก่อน: ถ้าไม่มี constraint จุด min ของ w^2 คือ $w=0$ — แต่ $w=0$ ละเมิด $w \geq 1$! ดังนั้น constraint ต้อง "กีด"

เขียนเงื่อนไข KKT:

- Stationarity: $2w + \mu(-1) = 0 \rightarrow 2w - \mu = 0$
- Complementary slackness: $\mu(1-w) = 0$

ลองเคส constraint active ($1-w=0 \rightarrow w=1$):

จาก stationarity: $\mu = 2w = 2(1) = 2 \rightarrow \mu=2 > 0 \checkmark$ (dual feasibility ผ่าน)

เช็ค primal: $w=1 \geq 1 \checkmark \rightarrow$ ทุกเงื่อนไขผ่าน $\rightarrow$ คำตอบ $w^* = 1, \mu = 2$

ตีความ $\mu=2$ เป็น shadow price: ถ้าผ่อน constraint จาก $w \geq 1$ เป็น $w \geq 0.9$ (ขยับพื้นลง 0.1) ค่า f จะลดลงประมาณ $\mu \times 0.1 = 2 \times 0.1 = 0.2$

(ตรวจสอบ: $f(1)=1, f(0.9)=0.81 \rightarrow$ ลดลง $0.19 \approx 0.2 \checkmark$)

เชื่อมกับ Options — Long-Only Markowitz

Markowitz แบบ long-only คือ: Minimize $w^T \Sigma w$ s.t. $\Sigma w = 1$ AND $w \geq 0$

constraint $\Sigma w = 1$ เป็น equality (ใช้ Lagrange λ) แต่ constraint $w \geq 0$ เป็น inequality — ต้องใช้ KKT (μ_i) จัดการ

กลไก: สินทรัพย์ที่ "อยาก" ได้น้ำหนักติดลบ (model อยาก short) จะโดน complementary slackness ตรึงไว้ที่ $w=0$ (active constraint, $\mu > 0$) ส่วนสินทรัพย์ที่เหลือมี $w > 0$ (inactive, $\mu=0$) ก็แก้ระบบ equality ตามปกติ

นี่คือเหตุผลที่ optimizer จริงใช้ Quadratic Programming (QP) ที่มี KKT ไม่ใช่ Lagrange ล้วนๆ — เพราะ Lagrange จัดการ "ห้าม short" ไม่ได้

⚠ **Model Risk: GIGO — Markowitz ดีแค่เท่ากับ Input ที่ป้อนเข้าไป**

"Garbage In, Garbage Out" — optimizer หา optimal solution ของ problem ที่ถูกตั้ง ไม่ใช่ problem จริง:

- **Expected return (μ):** ประมาณยากที่สุด — error เล็กน้อยทำ weight เปลี่ยนมาก (error maximization)
- **Covariance Σ :** unstable — Σ ปีที่แล้วอาจไม่ valid ปีนี้ โดยเฉพาะตอน regime change
- **Constraints ต้องสมเหตุสมผล:** KKT หา optimal ของ constraints ที่กำหนด — ถ้า constraints ผิด optimal ก็ผิด

แนวทางปฏิบัติ: Black-Litterman model (blend prior + views), cap weight ไม่เกิน 20%, rebalance ทุก quarter

11.5 Convex Optimization

อุปมา: ชาม vs อาน

Convex function = รูปชาม → มีก้นชามเดียว ไม่ว่าเริ่มจากจุดไหน Gradient Descent จะหาค้นหาเจอ

Non-convex = รูปอาน (saddle) หรือภูมิประเทศขรุขระ → มีหลายหุบเขา อาจติดอยู่ที่ "หุบเขาเล็ก" ที่ไม่ใช่ที่ดีที่สุด

ข่าวดีสำหรับ Quant: Portfolio optimization (Markowitz) เป็น **convex**!

ทำไม Convex สำคัญ?

- Convex → มี **minimum เดียว (global)** → Gradient Descent จะหาเจอเสมอ ไม่ว่าเริ่มจากไหน
- Non-convex → มีหลาย local minimum → อาจติดอยู่ที่จุดที่ไม่ใช่ global minimum
- การตรวจสอบ: f convex $\iff$ Hessian (matrix of second derivatives) เป็น PSD

ข่าวดี: Portfolio optimization (Markowitz) เป็น **convex**! → มีคำตอบเดียวที่ดีที่สุด

ข่าวร้าย: Calibration ของบาง model (เช่น Stochastic Volatility) เป็น **non-convex** → ต้องใช้เทคนิคพิเศษ

11.6 Quadratic Programming (QP)

ทำไม QP ถึงเป็น "ขั้นพิเศษ" ของ Nonlinear Optimization?

QP = objective เป็น quadratic (โด่งพาราโบลา) + constraints เป็น linear

นี่คือ "จุดกึ่งกลาง": ยากกว่า LP (เพราะ objective ไม่เส้นตรง) แต่ง่ายกว่า nonlinear ทั่วไป (เพราะ convex)

ข้อดี: มี solver ที่เร็วและแม่นยำ — แก้ Markowitz ได้ใน milliseconds!

$$\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} \quad \text{s.t. } \mathbf{A} \mathbf{x} \leq \mathbf{b}$$

$\mathbf{Q} = 2\Sigma$ (Covariance)

→

objective = $\mathbf{w}^T \Sigma \mathbf{w}$

→

constraints = budget + return

→

QP Solver

→

Efficient Portfolio!

เชื่อมกับ Options

Mean-Variance Optimization ของ Markowitz เป็น QP พอดี!

$\mathbf{Q} = 2\Sigma$ (covariance matrix), $\mathbf{c} = 0$ หรือ $-\mu$

ใช้ QP solver ใน Python (cvxpy, scipy) แก้ได้ในเสี้ยววินาที → optimal portfolio weights!

- 1 ตัวอย่าง QP 2 สินทรัพย์: หน้า 4, $\Sigma = \begin{bmatrix} 0.04 & 0.01 \\ 0.01 & 0.09 \end{bmatrix}$, $\mu = [0.10, 0.15]$
- 2 Minimize $\mathbf{w}^T \Sigma \mathbf{w} = w_1^2(0.04) + 2w_1w_2(0.01) + w_2^2(0.09)$
- 3 Subject to: $w_1 + w_2 = 1$, $0.10w_1 + 0.15w_2 = 0.12$ (target return)
- 4 จาก constraints: $w_2 = 0.4$, $w_1 = 0.6$ (แก้ระบบเส้นตรง)
- 5 $\sigma_p^2 = 0.36(0.04) + 2(0.24)(0.01) + 0.16(0.09) = 0.0144 + 0.0048 + 0.0144 = \mathbf{0.0336}$
- 6 $\sigma_p = \sqrt{0.0336} \approx \mathbf{18.3\%}$ ← portfolio risk ที่ optimal สำหรับ return 12%

แบบฝึกหัด Part V

โจทย์ 1: หา Optimal Solution ด้วย Corner Points

Maximize $z = 3x_1 + 2x_2$

Subject to: $2x_1 + x_2 \leq 10$, $x_1 + 2x_2 \leq 8$, $x_1, x_2 \geq 0$

หาทุก corner points และบอกว่า optimal solution คืออะไร?

► ดูเฉลย

โจทย์ 2: Shadow Price และ Sensitivity

จากตัวอย่าง Options ในบทเรียน (Maximize $8x_1 + 5x_2$ s.t. $5x_1 + 2x_2 \leq 20$, $x_1 + x_2 \leq 5$)

คำตอบ optimal คือ $x_1 = 10/3$, $x_2 = 5/3$, $z = 35$

ถ้า budget เพิ่มขึ้น B21 (จาก B20) คาดว่า Expected Payoff จะเป็นเท่าไร? (shadow price = 1)

► ดูเฉลย

โจทย์ 3: Gradient Descent Iterations

กำหนด $f(x) = x^2 + 4x + 5$, $\alpha = 0.1$, เริ่มที่ $x_0 = 3$

คำนวณ 3 iterations ของ Gradient Descent แล้วบอกว่าเข้าใกล้คำตอบจริงไหม?

► ดูเฉลย

โจทย์ 4: KKT Conditions กับ Inequality Constraint

Minimize $f(x) = (x-3)^2$ subject to $x \leq 2$

ใช้เงื่อนไข KKT หา x^* และ μ พร้อมบอกว่า constraint active หรือ inactive?

► ดูเฉลย

สรุป Part V

- ✓ LP: Objective + Constraints → Simplex → Optimal ที่มุม feasible region
- ✓ Dual: Shadow Price = มูลค่าของ constraint — constraint ไหน "แพง" ที่สุด?
- ✓ ILP: ตัวแปรจำนวนเต็ม → Branch & Bound (ใช้สำหรับ option contracts)
- ✓ Gradient Descent → เดินลงเขา → Model Calibration, Vol Surface Fitting
- ✓ Lagrange → Markowitz Portfolio Optimization → Efficient Frontier
- ✓ KKT → จัดการ inequality (ห้าม short $w \geq 0$, position limit) → Long-Only Markowitz
- ✓ Convex → มีคำตอบเดียว (global) ไม่ติด local minimum
- ✓ QP = Mean-Variance Optimization → แก้ด้วย cvxpy ใน milliseconds

จบ Part V

Part VI: Time Series, Numerical Methods & Stochastic Calculus

→ Forecast Volatility, IV Solver, ที่มาของ BS